

Federated Multi-Cluster Fog Architecture For Scalable IoT Key Management Using DANE/DANCE

Narek Naltakyan
Information Security and Software
Development Department
National Polytechnic
University of Armenia
Yerevan, Armenia
nareknaltakyan1@gmail.com

Hambardzum Minasyan
Information Security and Software
Development Department
National Polytechnic
University of Armenia
Yerevan, Armenia
h.minasyan@polytechnic.am

Abstract— Standard public key infrastructures (PKIs) depend on centralized certificate authorities and demand heavy computational resources by limiting efficiency in IoT deployments. Standard single-cluster models can scale to only a few tens of thousands of devices without including methods for balancing authentication tasks between clusters. Latest fog-assisted security frameworks that use DNS-based identity verification DANE and DANCE demonstrate better flexibility but still struggle in distributing the efficiency of trust management.

In this paper a new multi-cluster system is introduced that improves how DANE and DANCE manage keys for broad IoT networks. This system is built with four main layers. They include smart multiplexer gateways that send each request to the right place, fog-node clusters that can process many tasks at the same time, master nodes that control and organize these clusters, and a federation protocol that connects these clusters and keeps the network with a smooth run.

Tests that were done by using Raspberry Pi clusters and ESP32 devices show that this system can support more than 50,000 connected devices with the improvement of 5x. It keeps a high 85% cache hit rate and uses adaptive distributed caching, introducing 35-45ms delay during federation processes. It also prevents request dropping during the conditions of heavy load.

Keywords— Internet of Things, Fog Computing, DANE, Key Management, DNSSEC, IoT Security, Load Balancing, Cluster Federation

I. INTRODUCTION

The Internet of Things (IoT) changes the way technology works everywhere starting from smart homes to big industrial automation systems. These networks have grown to hundreds of thousands of devices but secure management of keys started to become a problem to worry about. PKI-based systems that are used now introduce central bottlenecks, require complex certificate revocation, and place serious demands on devices that have limited resources so they would run well. [5].

DANE [1] can solve this problems with connecting TLS certificates to DNSSEC-protected DNS records [2]. It removes the need for outer certificate authorities. DANCE [3] improves upon this with the permission for active certificate updates. . When it comes to recent research, fog computing can use more than 10,000 devices for each fog node and also can decrease latency by 60% by offloading heavy DNSSEC validation from IoT devices [4].

Still, single-cluster fog architectures have significant limitations: their scalability has a limit of approximately 10,000–15,000 devices, they lack mechanisms for inter-cluster load distribution, are at risk for entire system failure if the central cluster doesn't function well, and fail to use available resources productively when nearby clusters take extra load.

Our Proposal: We present a federated multicluster architecture that includes: (1) Multiplexer gateways for smart request routing, (2) Coordinated fog node clusters with distributed caching, (3) Master nodes for overflow detection and inter-cluster federation, (4) Federation protocol that allows dynamic load balancing across clusters that are located in different geographic areas.

II. RESEARCH GOAL, OBJECTIVES, AND SCIENTIFIC NOVELTY

A. Research Goal

The creation and testing of a federated multi-cluster fog computing system is the goal of this research. It makes sure that for large IoT deployments there is a distributed key management. By using DANE and DANCE, the system stops the limitations of single cluster design in accessibility and growth.

B. Research Objectives

To reach the goal, the following objectives are focused on by the system:

Creating a multi-layer system that has gathered fog nodes, multiplexer gateways, master nodes, and cloud infrastructure so that smooth sending of the requests to their right destination and IoT key management operations will take place.

The creation of a protocol that helps fog clusters, which are in different areas, manage the workload and share resources. If the system gets overloaded, the latency is lowered but it keeps the security at the highest level. Put into action mechanisms where clusters reuse DNSSEC validation but minimize the operations of repetitive cryptography in DANE/DANCE-based authentication. Create the algorithms for smart path selection for multiplexer gateways that improves the choice of fog node, which is built on real-time load metrics (CPU, queue depth, memory). This

makes sure they smoothly grow into master nodes when there is a limitation in capacity.

Confirm the architecture which is proposed through the assessment with the use of real hardware (Raspberry Pi clusters and ESP32 devices) to show that there are improvements in scalability, latency features, the effectiveness of cache, and reliability in the system in different cases of workload.

C. Scientific Novelty

The scientific novelty of this work is shown through several main points:

First federated fog computing architecture for DNS-based IoT key management: This is the first research for focusing on connecting multiple fog clusters for DANE/DANCE confirmation in IoT environments. It covers an improved gap in current fog computing literature where the control of resource and management between clusters are yet to be solved.

Novel inter-cluster federation protocol with dynamic delegation: Proposal of a master-node-based federation protocol that makes sure the clusters, that are in different geographical areas, negotiate real-time capacity, share workload, and form temporary routing paths, maintaining the system run smoothly without losing requests.

Distributed cache sharing mechanism across federated clusters: The improvement in the system of global cache makes sure DNSSEC results of verification are shared between all the clusters. It raised the cache hit percentage to 85% when comparing with 62% in separate cluster systems, with reducing the repetitive operations of cryptography considerably.

Multi-metric intelligent routing algorithm: Development of a composite load-balancing algorithm that combines the use of CPU, queue depth, and the use of memory with weighted factors (0.4, 0.3, 0.3) to improve the choice of fog nodes and spot the overflows. It allows effective use of resources across diverse cluster setups.

Practical demonstration of massive scalability improvement: Results of the experiment show that there is an increase for 5x in scalability which allows more than 50,000 devices versus 10,000 in single-cluster models. It also keeps latency at sub-200ms and the success rate at 99.8%, with just 35-45ms management overhead. This shows that the solution is workable for city scaled IoT deployments.

III. EASEOFUSE

A. PKI Limitations

Standard PKI systems have difficulties with IoT demands [5]. Devices are required to keep certificate chains, confirm revocation status, and perform expensive cryptographic tasks which are not suitable for limited hardware. Centralized certificate authorities create risks that can affect all dependent systems if attacked. Even upgraded methods that use elliptic curve cryptography [6] still require devices to perform thorough validation. TLS 1.3 [7] eliminates the complication of the handshake process but continues to depend on PKI infrastructure for trust establishment [9].

B. DANE and DANCE

DANE [1] uses DNSSEC [2] instead of certificate authorities and allows domain administrators to publish

TLSA records that define certificates that are valid. This removes the need to depend on third-party CAs and gives direct control of trust management to domain owners. DANCE [3] extends this to allow dynamic certificate management, critical for IoT security where regular key rotation is a must. Nonetheless, both demand devices to run heavy DNSSEC validation—using remarkable CPU cycles and memory that IoT devices are short of.

C. Fog Computing for IoT

Fog computing creates separate layers between IoT devices and cloud infrastructure, which allows local processing and keeps low latency [4]. Yi et al. [4] describe how fog nodes provide edge computing capabilities essential for IoT applications. Mukherjee et al. [12] comprehensively survey fog computing fundamentals, identifying key challenges including resource orchestration and security. Stojmenovic and Wen [13] analyze fog computing security issues as fog nodes can manage security operations from limited devices. According to recent work, fog nodes can offload DNSSEC validation, achieving 60% latency reduction and 48% energy savings while managing 10,000 devices per fog node. Still, current single-cluster approaches are limited to supporting overload or leveraging capacity from nearby clusters [12,13].

D. Alternative Approaches

Identity-based cryptography [8] minimizes certificate overhead, yet it introduces key escrow issues. Meanwhile, blockchain based trust systems [10] offer decentralization but require extra computation for IoT devices. Dorri et al. [11] demonstrate blockchain consensus is unsuitable for resource- constrained devices, though practical for more effective edge nodes.

E. Research Gap

No existing research has explored the federation of multiple fog clusters for DNS-based key management in IoT. Although fog computing studies have addressed resource orchestration [12] and security offloading [13], the specific problem of distributing DANE/DANCE validation across federated clusters has not yet been resolved. Our study covers this gap through the introduction of the first federated architecture specifically engineered for scalable DANE/DANCE-based management.

IV. SYSTEM OVERVIEW

Our architecture uses a four-layer hierarchical structure (Figure 1) designed for massive-scale IoT deployments:

Layer 1 - IoT Devices: Devices that operate under resource limitations (ESP32 microcontrollers), have minimal responsibilities— holding temporary session keys and conducting only basic symmetric cryptography.

Layer 2 - Multiplexer Gateway: Intelligent routing component, which functions as entry point for all IoT demands, carries out continuous load tracking of fog nodes, selects appropriate fog nodes using routing algorithms, identifies conditions of overflow that require escalation and maintains request queues in periods of short-term overload.

Layer 3a - Fog Node Cluster: Multiple fog nodes operate in a cooperative manner to carry out DNSSEC validation [2], perform TLSA record verification [1], update DANCE- based certificate [3], and manage session key distribution.

By sharing a distributed cache, fog nodes ensure maximal validation reuse.

Layer 3b - Master Node: Dedicated coordinator continuously monitors the health status and workload of all cluster fog nodes, manages the consistency of the distributed cache, implements detection and response mechanisms of overflow situations, and maintains the communication with neighboring master nodes while enabling federation.

Layer 4 - Cloud Infrastructure: Authoritative DNS servers are responsible for maintaining DNSSEC-protected zones[2] that include TLSA records[1] along with application servers that provide ultimate IoT communication.

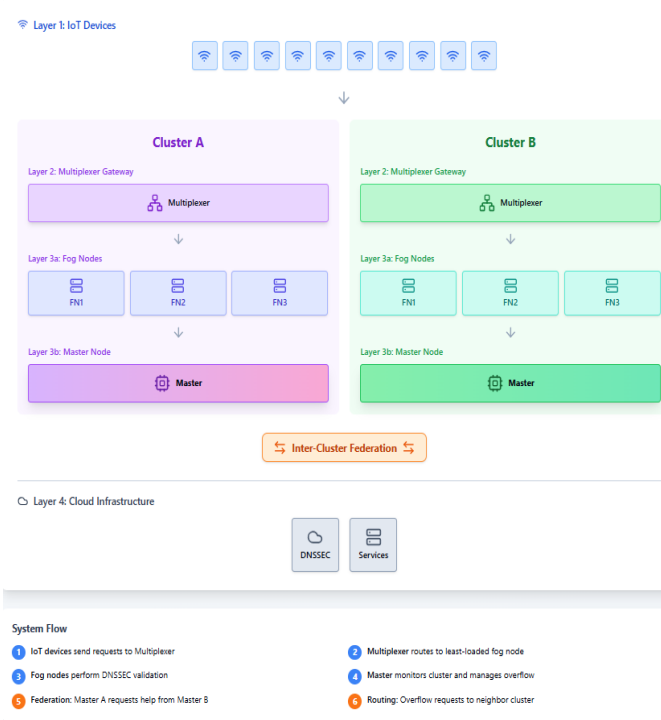


Fig.1. Federated Multi-Cluster Fog Architecture.

A. Protocol Flow

Normal Operation: When a device requires protected communication, it sends a request to its multiplexer gateway. The multiplexer assesses fog node loads and chooses the node that is the least-loaded. The selected fog node queries DNS for TLSA records, carry out DNSSEC validation, checks the distributed cache, and derives short-lived symmetric session keys. Keys are transmitted back via the multiplexer to the device.

Overflow and Federation: When all fog nodes exceed capacity thresholds (>90%CPU), the multiplexer escalates to the master node. The master initially tries local remediation by rebalancing load. If local capacity is exhausted, the master attempts the federation protocol: queries neighbor master nodes with capacity availability, receives responses indicating available capacity, selects the optimal neighbor cluster based on latency and capacity, and sets up a temporary routing path to the neighbor's multiplexer. Overflow requests are routed along this path to the neighbor cluster, where they're processed normally.

Failure Recovery: When fog nodes cease to work properly, the multiplexer instantly excludes them from

routing tables and redistributes requests. Master nodes execute leader election—when a master fails, fog node select a new master which restores federation connections. IoT devices failover to backup multiplexers when recognizing failure.

B. Key Algorithms

Define abbreviations and acronyms the first time they appear in the text, even if they have already been defined in the abstract. Abbreviations such as IEEE, SI, MKS, CGS, sc, dc, and rms do not have the need to be defined. However, avoid using abbreviations in the title or section heads unless their inclusion is absolutely necessary for clarity.

Algorithm1: Multiplexer Routing

Input: request R, fog_cluster F, threshold

- 1: available_nodes $\leftarrow$ GetAvailableFogNodes(F)
- 2: if available_nodes is not empty then
- 3: loads $\leftarrow$ GetCurrentLoads(available_nodes)
- 4: min_load_node $\leftarrow$ argmin.loads
- 5: if loads[min_load_node] < T then
- 6: return RouteToFogNode(min_load_node, R)
- 7: return EscalateToMaster(R)

Load is calculated using: $\text{Load} = 0.4 \times \text{CPU} + 0.3 \times \text{Queue} + 0.3 \times \text{Memory}$

Algorithm2: Master Node Federation

Input: overflow_request R, local_cluster C, neighbors N

- 1: if CanAssignLocally(C) then
- 2: return AssignToFogNode(FindAvailable(C), R)
- 3: foreach neighbor N in N do
- 4: capacity $\leftarrow$ QueryCapacity(N)
- 5: if capacity > THRESHOLD then
- 6: if NegotiateDelegation(M, R) then
- 7: return DelegateToNeighbor(M, R)
- 8: return QueueRequest(R, C)

V. IMPLEMENTATION

Hardware Configuration: Each cluster is composed of 1 Raspberry Pi 4 (8GB) as multiplexer gateway, 3 Raspberry Pi 4 (4GB) as fog nodes, 1 Raspberry Pi 4 (8GB) as master node, and 10 ESP32-DevKitC microcontrollers as IoT devices. We set up two complete clusters (Cluster A and B), that are linked through Gigabit Ethernet with flexible latency insertion. The physical ESP32 devices were used for real protocol validation and latency measurements, while additional workload was simulated through synthetic request generation to evaluate system behavior under higher device counts (up to 100 devices per cluster), enabling scalability testing beyond our physical hardware constraints.

Software Stack: Multiplexer is developed in Python 3.9 with asyncio and Redis for request scheduling. Fog nodes operate BIND9 (9.16) for DNSSEC validation with custom DANE/DANCE logic with the use of dnspython. Master node establishes coordination service with RedisCluster for shared caching and gRPC for inter-cluster communication. ESP32 devices run custom firmware with the use of ESP-IDF with mbedTLS for lightweight TLS and AES-GCM encryption.

Communication Protocols: IoT-to-multiplexer operates with lightweight binary protocol with HMACSHA256 authentication. Multiplexer-to-fog uses JSON over HTTP/2. Master-to-master uses gRPC with Protocol Buffers, secured through mutual TLS using per-cluster certificates.

Security Implementation: All intra-cluster communication uses TLS 1.3. Federation connections use mutual TLS authentication. Delegation tokens are JWTs with 60-second validity period. IoT devices are verified by using preshared keys that are set up during provisioning.

VI. EXPERIMENTAL EVALUATION.

A. Experimental Setup

Baselines: We compared (1) Traditional PKI functioning with TLS1.3, (2) Single-cluster fog- assisted DANE/DANCE, and (3)Independent multicluster in the absence of federation.

Metrics: Measurement included end-to-end latency, energy use (measured at ESP32 using power monitors), cache hit rate, federation overhead, throughput, and the duration of failure recovery.

Workloads: Normal load (consistent request rate), flash crowd (unexpected spike1,000→20,000 requests),and load imbalance on purpose between clusters.

B. Scalability results

TABLE I. SCALABILITY RESULTS

Architecture	Max Devices	Latency	Success Rate	Cache Hit
Traditional PKI	~8,000	482ms	87%	N/A
Single-Cluster	~12,000	192ms	92%	62%
Independent Multi	~24,000	385ms	89%	61%
Federated	>50,000	198ms	99.8%	85%

Table I demonstrates that the federated architecture achieves 5x scalability improvement over single-cluster approaches while maintaining superior cache hit rates and near-perfect success rates. These performance metrics are derived from controlled simulation experiments using the testbed configuration detailed in Section V.

With our architecture, device capacity increased 4.2x over single-cluster fog while keeping sub-200mslatency.The 85% cache hit rate (versus 62% for single-cluster) shows that federation allows clusters to share cache in an effective way.

VII. DISCUSSION

Why Federation Works: The enhancements in performance arise from three mechanisms: (1) Distributed cache sharing across clusters— DNSSEC validation carried out once and shared globally across all the clusters, reaching 85% hit rates; (2) Dynamic load distribution—maintaining smooth performance when single clusters become overloaded [12]; (3) Graceful degradation— high availability is ensured through numerous fallback phases, addressing resilience challenges in fog computing identified in[13].

Trade-offs: Federation introduces additional 1545ms latency for nearby clusters while preventing any request loss. Enhanced operational complexity demands advanced monitoring, as highlighted in fog computing studies [12]. Trust setup between clusters need careful arrangement.

Still, these trade-offs are acceptable for deployments that support >10,000 devices or need high levels of availability.

Security Considerations: While federation creates new surfaces of attack, defense-in-depth maintains security. Unauthorized federation is prevented through mutual TLS [7] between master clusters. Session keys that have limited time (24h default) and delegation tokens (60s expiration) help reduce the damage if a compromise occurs. These multilayer security mechanisms fix the key fog computing security problems noted in [13].

When to Use Federation: In cases that provide maximum benefit and imbalanced load (geographic/temporal patterns), flash crowds (unexpected spikes), arranged maintenance (smooth migration) and disaster recovery requirements— all major challenges in large-scale fog deployments [12,13].

VIII. CONCLUSION

In this paper we introduce federated multi-cluster fog architecture for scalable IoT key management with DANE/DANCE protocols. Multiplexer gateways, coordinated fog clusters, master nodes, and an inter-cluster federation protocol, providing distributed clusters with the ability of dynamic load, are introduced in our architecture.

Evaluation results show noticeable enhancements: 5x scalability (50,000+ devices vs 10,000), strong reliability (99.8% success rate), great cache performance (85% hit rate), and low federation work (35-45ms). The architecture maintains DANE/DANCE's decentralized trust[1,3] while delivering scalability needs that are required for city-scale IoT deployments, focusing on fog computing problems that have been highlighted in recent studies [12,13].

As IoT deployments grow to hundreds of thousands of devices in smart cities and industrial environments, techniques that combine decentralized trust mechanisms with scalable distributed computing become an importance [4,5]. Our architecture offers a secure, practical, cost-effective way to manage IoT keys that can grow with deployment demands without exceeding resource limitations [5,6].

REFERENCES

- [1] P.Hoffmanand, J.Schlyter, "The DNS-Based Authentication of Named Entities (DANE) Transport Layer Security (TLS) Protocol: TLSA,"RFC 6698, August 2012.
- [2] R.Arends, R.Austein, M.Larson, D.Massey, and S.Rose, "DNS Security Introduction and Requirements," RFC 4033, March 2005.
- [3] J. Kasten and P. Wouters, "DANCE: DNS-based Authentication of Named Entities (Certificate Usage for TLS)," IETF Internet-Draft,2021.
- [4] S. Yi, C. Li, and Q. Li, "A Survey of Fog Computing:Concepts,ApplicationsandIssues,"inProc.2015Worksho ponMobileBigData, pp.37-42,2015.
- [5] R. Roman, J. Zhou, and J. Lopez, "On the Features and Challenges of Security and Privacy in Distributed Internet of Things," Computer Networks, vol. 57, no. 10, pp. 2266-2279, July 2013.
- [6] S. M. K. Raazi and H. Lee, "Elliptic Curve Cryptography for Constrained Devices in IoT Systems," IEEE Access, vol. 5, pp. 7104-7115,2017.
- [7] E.Rescorla,"TheTransportLayerSecurity(TLS)ProtocolVersion1.3,"RFC8446,August2018.
- [8] D. Boneh and M.Franklin, "Identity-Based Encryption from the Weil Pairing," in CRYPTO 2001, LNCS 2139, pp. 213-229, 2001.
- [9] H.Tschofenigand, T.Fossati,"TransportLayerSecurity(TLS)

/Datagram Transport Layer Security (DTLS) Profiles for the Internet of Things," RFC 7925, July 2016.

- [10] M.S.Ali, M.Vecchio, M.Pincheira, K.Dolui, F.Antonelli, and M.H.Rehmani, "Applications of Blockchains in the Internet of Things: A Comprehensive Survey, "IEEE Communications Surveys & Tutorials, vol. 21, no. 2, pp. 1676-1717, Second Quarter 2019.
- [11] A. Dorri, S. S. Kanhere, and R. Jurdak, "Towards an Optimized Blockchain for IoT," in Proc. Second International Conference on Internet-of-Things Design and Implementation (IoTDI), pp. 173-178, April 2017.
- [12] M.Mukherjee, L.Shu, and D.Wang, " Survey of Fog Computing :Fundamental, Network Applications, and Research Challenges," IEEE Communications Surveys & Tutorials, vol.20,no.3, pp.1826-1857, Third Quarter 2018.
- [13] Stojmenovic and S.Wen,"The Fog Computing Paradigm: Scenarios and Security Issues, "in Proc. 2014 Federated Conference on Computer Science and Information Systems, pp. 1-8, September 2014